COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/16//2026	**DATE SUBMITTED:**	09/16//2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 9.1 Implementing Trees

1. PROCEDURE OUTPUT

What procedures did you perform?

```cpp
#include <iostream>
#include "binaryTree.h"

int main(){

    // create a root of the node
    BinaryTree<char>* root = new BinaryTree<char>('A');

    // create a child node
    root->insertLeftChild('B');
    root->insertRightChild('C');

    // create a subchild of the node
    root->getLeftChild()->insertLeftChild('D');
    root->getLeftChild()->insertRightChild('F');

    root->getRightChild()->insertLeftChild('E');

    // print the root node
    std::cout << "Root value: " << root->getKey() << std::endl;
    std::cout << "Left child of root: " << root->getLeftChild()->getKey()
    << std::endl;
    std::cout << "Right child of root: " << root->getRightChild()->getKey
    () << std::endl;

    std::cout << "Left child of root: " << root->getLeftChild()
    ->getLeftChild()->getKey() << std::endl;
    std::cout << "Right child of root: " << root->getLeftChild()
    ->getRightChild()->getKey() << std::endl;

    // change the root of F
    root->getLeftChild()->getRightChild()->changeKey('K');
    std::cout << "Right child of root: " << root->getLeftChild
    ()->getRightChild()->getKey() << std::endl;

    // test leaf
    std::cout << "Is Right child of root a leaf: " <<
    root->getLeftChild()->getRightChild()->isLeaf() <<
    std::endl;

    std::cout << "Is Left child a leaf: " << root->getLeftChild
    ()->isLeaf() << std::endl;

    return 0;
}
```

Figure 1. Main Driver Code of Binary Tree.

This main driver code contains the process in creating a binary tree using the functions in the header file to perform various operations, such as creating a root of the node, creating its child node, reassigning the root value, and printing the values inside the nodes. There is also a check to determine if the assigned node to be checked is a leaf or not. The isLeaf function simply checks if the said node has a subnode or not by seeing whether its node heads to the null pointer. Overall, this driver code tests the working functions of the binary tree by calling each function and connecting them to form a binary tree.

```
#ifndef BINARYTREE_H
#define BINARYTREE_H

template<typename T>
class BinaryTree{
    private:
        T key;
        // pointer
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:
        // constructor
        BinaryTree(T rootObj);

        // add child to node
        void insertLeftChild(T newNode);
        void insertRightChild(T newNode);
        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();
        T getKey();
        void changeKey(T val);
        bool isLeaf();
};

template<typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}
```

```
33  template<typename T>
34  void BinaryTree<T>::insertLeftChild(T newNode){
35      if (leftChild == nullptr){
36          leftChild = new BinaryTree<T>(newNode);
37      } else {
38          BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
39          // move the child under the new node
40          newChild->leftChild = leftChild;
41          leftChild = newChild;
42      }
43  }
44
45  template<typename T>
46  void BinaryTree<T>::insertRightChild(T newNode){
47      if (rightChild == nullptr){
48          rightChild = new BinaryTree<T>(newNode);
49      } else {
50          BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
51          // move the child under the new node
52          newChild->rightChild = rightChild;
53          rightChild = newChild;
54      }
55  }
56
57  template<typename T>
58  BinaryTree<T>* BinaryTree<T>::getLeftChild(){
59      return leftChild;
60  }
61
62  template<typename T>
63  BinaryTree<T>* BinaryTree<T>::getRightChild(){
64      return rightChild;
65  }
```

```
67  template<typename T>
68  T BinaryTree<T>::getKey(){
69      return key;
70  }
71
72  template <typename T>
73  void BinaryTree<T>::changeKey(T val){
74      key = val;
75  }
76
77  template<typename T>
78  bool BinaryTree<T>::isLeaf(){
79      return leftChild == nullptr && rightChild == nullptr;
80  }
81
82  #endif
```

Figure 2. Header Code of Binary Tree.

This header code of the binary tree contains eight functions that perform the basic operations in a binary tree, which are the get child, insert child, get key, change key, and leaf check functions used in the main driver code to create a functional binary tree. The class contains the function declarations, pointer variables, and the constructor containing the root object. This constructor has the key, left, and right child, which are used to form the base tree. The insert child functions have an if statement to check whether it leads to a null pointer or called a leaf, otherwise, it creates a new child in a new node under it. The get functions simply return the respective child and key variables. The change key function reassigns the key value to the value assigned in the function call in the driver code to change the value, which is seen in the output in the next figure. The isLeaf function is a simple check function to determine if the node has no child or a null pointer at its end.

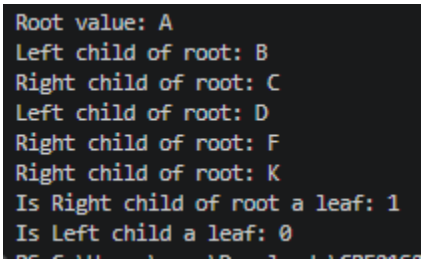


Figure 3. Output Console of Binary Tree.

This console output screenshot contains the printed values in the driver code as seen in the first figure. The printed values come from the get child function calls that take the value of the created child nodes. To form the binary tree, it requires creating the initial child node with their own values, and another subchild of the node to create a subtree. The get key functions are used in printing the key value of a certain child in the binary tree. Looking at the change of root value at K, a change key function is used to reassign the key value from F to K in the selected node, which is the right subchild of the left child of the tree. The leaf check is a boolean result that checks if the child ends with a null pointer or leads to another subchild.

Conclusion:

This seatwork activity has demonstrated the applications of binary trees using constructors, insertion, get child and key functions, and change key functions to manipulate the values stored in the tree. In the driver code, the root is always the starting point of a binary tree before going down to the left and right child, and then the other subtrees of the child. This is a form of hierarchical data structure, which is composed of connected trees of nodes and subnodes. The functions in the header perform the basic tree operations of adding another node, obtaining the value of the node, and reassigning the value of a certain node. Leaf child nodes are nodes that contain no child and end with a null pointer. Overall, this activity teaches the fundamental understanding of hierarchical data structures by applying the concepts of binary trees into a working implementation with appropriate functions that show the algorithms that are responsible for making the binary tree work.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.